

AI Report for Lab 3

3.3.1 Technology Stack Prompting

Prompt

We provided the AI with our Lab 1 system description and our updated class diagram for **Smart Commute Optimizer**, a mobile application that recommends 2 to 3 commute routes based on user inputs such as origin, destination, and departure time, together with user preferences such as fastest, most reliable, or most comfortable. The system uses real-time transport data, including MRT crowding and wait time, taxi availability, and carpark availability, and includes route scoring, delay and crowding risk estimation, and explainable route recommendations.

We then asked the AI to recommend a suitable technology stack for implementing the updated design, with justification for the programming language, frontend, backend, database, supporting tools, and project structure.

AI Response Summary

The AI recommended using TypeScript as the main programming language, React Native with Expo for the mobile frontend, NestJS for the backend, and PostgreSQL with Prisma for persistent storage. It also suggested a layered modular architecture, with separate modules for features such as trips, preferences, location, routes, scoring, and transport data. In addition, it recommended supporting libraries such as React Navigation, Expo Location, React Query, Zod, Axios, and i18next to handle navigation, GPS, API communication, validation, and localisation.

3.3.2 Comparison with Previous Lab

The AI's recommendations are mostly the same as in the previous lab, rather than exactly the same or completely different. In the earlier exercise, the recommended stack was also centred around React Native + TypeScript on the client side and Node.js + TypeScript with NestJS on the backend, together with PostgreSQL and Prisma for structured

persistent data. The previous recommendation also included Redis for caching transport snapshots, BullMQ for background refresh jobs, Jest and Supertest for testing, and Docker and GitHub Actions for deployment and collaboration support.

The main similarity is that both recommendations recognise the same core system characteristics. In both cases, the application is treated as a mobile-first route planning system with structured domain entities, real-time transport data integration, user preferences, route ranking, and the need for maintainable separation between frontend, backend, and persistent data. Both responses therefore converge on a modern TypeScript-based full stack, which is practical for a student team and maps well to the application's use cases.

The differences are mostly in level of detail and emphasis, not in overall direction. The previous lab response placed more attention on infrastructure and operational support, including Redis, BullMQ, Pino, Sentry, Docker, and GitHub Actions. By contrast, the new response focuses more directly on the updated class diagram, highlighting how domain classes, enums, controllers, and service interfaces can map naturally into TypeScript models and NestJS modules. It also recommends more frontend-specific libraries such as React Navigation, Expo Location, React Query, and i18next because the updated design makes user interaction, GPS handling, refresh behaviour, and localisation needs more explicit.

A likely reason for this shift is that the updated class diagram provides a clearer view of the system's internal structure. The new AI response explicitly refers to typed entities such as `TransportMode`, `OptimisationCriterion`, `RiskCategory`, `DataSourceType`, `TripRequest`, `RouteOption`, `RouteSegment`, and `UserPreferenceProfile`, as well as controller-like classes such as `TripController`, `LocationController`, `RouteScoringController`, and `DataFetchController`. This likely caused the AI to give a more design-driven recommendation, rather than a broader system-level recommendation.

Overall, I agree with the newer recommendation. It remains consistent with the previous lab's result, which suggests that the design direction is stable. At the same time, it is more tailored to the updated class diagram and therefore more useful at this stage of the project. The choice of TypeScript, React Native with Expo, NestJS, PostgreSQL, and Prisma is appropriate because it supports strong type consistency, modular design, and manageable implementation effort for a student software engineering team.

